Лабораторна робота №10

Семенюка Владислава Григоровича

Група ІПЗ-21

Варіант 7

Код консольної програми

```
using System.Security.Cryptography;
using Lab10Models;

class Program
{
    static void Main()
    {
        const string divider =
"-----------------------------------------------
--------------";
        const int variant = 7;

        Console.WriteLine(divider);
        Console.WriteLine("Lab №10");
        Console.WriteLine("Fullname: Vlad
Semeniuk");
        Console.WriteLine("Group: IPZ-21");
        Console.WriteLine($"Variant: {variant}");
        Console.WriteLine("Data Type:
BuildingEntry");
        Console.WriteLine(divider);

        var catalog = new Catalog<BuildingEntry>();

        var buildingEntry1 = new
BuildingEntry($"B-{variant}-01", "Потебні, 56",
"навчальний корпус", 5);
```

```
        var buildingEntry2 = new
BuildingEntry($"B-{variant}-02", "Лесі Українки
3", "офіс", 1 + variant % 10);
        var buildingEntry3 = new
BuildingEntry($"B-{variant}-03", "Львівська 75",
"бібліотеку", 2);

        catalog.Add(buildingEntry1);
        catalog.Add(buildingEntry2);
        catalog.Add(buildingEntry3);

        Console.WriteLine($"Count:
{catalog.Count}");
        Console.WriteLine();
        catalog.PrintAll();
        Console.WriteLine($"First item:
{catalog.GetAt(0)}\n");

        Console.WriteLine($"Contains first code:
{catalog.ContainsCode($"B-{variant}-01")}");
        Console.WriteLine($"Contains second code:
{catalog.ContainsCode($"B-{variant}-02")}");
        Console.WriteLine($"Contains third code:
{catalog.ContainsCode($"B-{variant}-03")}");
        Console.WriteLine($"Contains for invalid
code: {catalog.ContainsCode($"B-{variant}-04")}");
        Console.WriteLine(divider);

        Queue<BuildingEntry> queue = new
Queue<BuildingEntry>();
        Stack<BuildingEntry> stack = new
Stack<BuildingEntry>();

        //Add item for the queue
        queue.Enqueue(buildingEntry1);
```

```
        queue.Enqueue(buildingEntry2);
        queue.Enqueue(buildingEntry3);

        //Add item for the stack
        stack.Push(buildingEntry1);
        stack.Push(buildingEntry2);
        stack.Push(buildingEntry3);

        //Queue manipulations
        Console.WriteLine("Queue manipulations\n");
        Console.WriteLine($"Count: {queue.Count}");
        Console.WriteLine($"Peek:
{queue.Peek().Code}");
        Console.WriteLine($"Dequeue:
{queue.Dequeue().GetInfo()}");
        Console.WriteLine($"Count:
{queue.Count}\n");
        Console.WriteLine("All elements in the
queue after dequeue:\n");

        foreach (var item in queue)
        {
            Console.WriteLine($"{item.GetInfo()}");
        }

        Console.WriteLine(divider);

        //Stack manipulations
        Console.WriteLine("Stack manipulations\n");
        Console.WriteLine($"Count: {stack.Count}");
        Console.WriteLine($"Peek:
{stack.Peek().GetInfo()}");
        Console.WriteLine($"Pop:
{stack.Pop().GetInfo()}");
```

```
        Console.WriteLine($"Count:
{stack.Count}\n");
        Console.WriteLine("All elements in the
stack after pop:\n");

        foreach (var item in stack)
        {
            Console.WriteLine($"{item.GetInfo()}");
        }

        Console.WriteLine(divider);

        // Exp. The different between FIFO and LIFO
is that in FIFO - (First in first out)
        // So it acts like a real queue. We could
image a real queue in a shop so the people
        // stand one after another so the first get
his purposes then the next one gets
        // LIFO - (Last in first out) is a complete
opposite. We could image a stack of plates so we
washed them
        // so after some time we want to put them
in a wardrobe so we take the upper one and remove
        // them one after another.

        // Dictionary manipulations
        Dictionary<string, BuildingEntry?> dict =
new Dictionary<string, BuildingEntry?>();

        dict.Add($"B-{variant}-01",
buildingEntry1);
        dict.Add($"B-{variant}-02",
buildingEntry2);
        dict.Add($"B-{variant}-03",
buildingEntry3);
```

```
        Console.WriteLine("Dictionary
manipulations:\n");
        Console.WriteLine($"Dict Count:
{dict.Count}");


        if (dict.TryGetValue($"B-{variant}-01", out
BuildingEntry? entry1))
        {
            Console.WriteLine($"Found:
{entry1?.GetInfo()}");
        }


Console.WriteLine(dict.TryGetValue($"B-{variant}-0
4", out BuildingEntry? entry2)
            ? $"Found: {entry2?.GetInfo()}"
            : $"Not found for the B-{variant}-04
code");

        Console.WriteLine(divider);

        // Exp. Keys must be unique because
dictionary works in this way
        // List has unique indexes, dictionary has
identical keys
        // if dictionary had not unique keys the
whole logic would fail
        // because it works like safes each safe
stores its value and has unique key

        // Individual assignment A
```

```
        Console.WriteLine($"Remove for
B-{variant}-01:
{catalog.RemoveByCode($"B-{variant}-01")}");
        Console.WriteLine($"Remove for invalid
B-{variant}-04:
{catalog.RemoveByCode($"B-{variant}-04")}");
        Console.Write($"Count: {catalog.Count}");

        // Exp. foreach will not work for our
RemoveByCode method as we need to access
        // index for removing the right item
because we use RemoveAt which needs index.

    }
}
```

```
namespace Lab10Models;


public class BuildingEntry : IEntry
{
    public string Code { get; }
    public string Address { get; }
    public string Purpose { get; }
    public int FloorCount { get; }

    public BuildingEntry(string code, string
address, string purpose, int floorCount)
    {
        Code = code;
        Address = address;
        Purpose = purpose;
        FloorCount = floorCount;
    }
```

```
    public string GetInfo()
    {
        return $"Code: {this.Code}, address —
{this.Address}, purpose: {this.Purpose},
floorCount: {this.FloorCount}";
    }

}
```

```
namespace Lab10Models;
using System.Collections.Generic;


public class Catalog<T> where T : IEntry
{
    private readonly List<T> _items = new
List<T>();


    public int Count => _items.Count;


    public void Add(T item)
    {
        _items.Add(item);
    }


    public T GetAt(int index)
    {
        return _items[index];
    }
```

```
    public bool ContainsCode(string code)
    {
        foreach (T item in _items)
        {
            // Обмеження IEntry гарантує наявність
властивості Code.
            if (item.Code == code)
            {
                return true;
            }
        }
        return false;
    }

    public bool RemoveByCode(string code)
    {
        for (var i = 0; i < _items.Count; i++)
        {
            if (_items[i].Code != code) continue;
            _items.RemoveAt(i);
            return true;
        }

        return false;
    }


    public void PrintAll()
    {
        foreach (T item in _items)
        {
            // Обмеження IEntry гарантує наявність
GetInfo().
            Console.WriteLine(item.GetInfo());
```

```
        }
    }

}
```

```
namespace Lab10Models;


public interface IEntry
{
    string Code { get; }
    string GetInfo();
}
```

ШІ використовувався тільки для ресьорчу.